

QUANTUM-ENHANCED MULTI-MODAL VECTOR SEARCH: A HYBRID DATABASE WEB APP



Elhan Mahmutović

Adna Kikanović

Adnan Mušanović

Mirza Abdulahović

INTERNATIONAL UNIVERSITY OF SARAJEVO

2026

QUANTUM-ENHANCED MULTI-MODAL VECTOR SEARCH: A HYBRID DATABASE WEB APP

BY

Elhan Mahmutović – Software Engineering

Adna Kikanović – Software Engineering

Adnan Mušanović – Software Engineering

Mirza Abdulahović – Software Engineering

A graduation project submitted in partial fulfillment of the
requirements for the degree of Bachelor of Science at the
Faculty of Engineering and Natural Sciences
International University of Sarajevo

June 2026

APPROVAL PAGE

I certify that I have supervised and read this study and that in my opinion, it conforms to acceptable standards of scholarly presentation and is fully adequate, in scope and quality, as a graduation project for the degree of Bachelor of Science at the Faculty of Engineering and Natural Sciences.

.....
Assist. Prof. Dr. Ali Abd Almisreb

Mentor

DECLARATION

We hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. We also declare that, as required by these rules and conduct, we have fully cited and referenced all material and results that are not original to this work.

Elhan Mahmutović

Signature

Adna Kikanović

Signature

Adnan Mušanović

Signature

Mirza Abdulahović

Signature

Date

INTERNATIONAL UNIVERSITY OF SARAJEVO

DECLARATION OF COPYRIGHT AND AFFIRMATION OF FAIR USE OF UNPUBLISHED WORK

Copyright © 2026 by Elhan Mahmutović, Adna Kikanović, Adnan Mušanović, and Mirza Abdulahović. All rights reserved.

Quantum-Enhanced Multi-Modal Vector Search: A Hybrid Database Web App

No part of this unpublished work may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise without prior written permission of the copyright holder and IUS Library.

Affirmed by:

Elhan Mahmutović

Signature

Adna Kikanović

Signature

Adnan Mušanović

Signature

Mirza Abdulahović

Signature

Date

ACKNOWLEDGEMENTS

We thank our supervisor, Assist. Prof. Dr. Ali Abd Almisreb, for his guidance, patience, and feedback throughout this project. We are grateful to the Faculty of Engineering and Natural Sciences at the International University of Sarajevo for the teaching and resources that made this work possible. Finally, we thank our families and friends for their encouragement and support.

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CLIP	Contrastive Language – Image Pre-training
CPU	Central Processing Unit
FAISS	Facebook AI Similarity Search
GPU	Graphics Processing Unit
HNSW	Hierarchical Navigable Small World
IBM QPU	IBM Quantum Processing Unit
MRR	Mean Reciprocal Rank
NISQ	Noisy Intermediate-Scale Quantum
qRAM	Quantum Random-Access Memory
SDG	Sustainable Development Goal
SQL	Structured Query Language
ViT	Vision Transformer

TABLE OF CONTENTS

APPROVAL PAGE	iii
DECLARATION	iv
DECLARATION OF COPYRIGHT	v
ACKNOWLEDGEMENTS	vi
LIST OF ABBREVIATIONS	vii
ABSTRACT	x
1 INTRODUCTION	1
1.1 Background of Study	1
1.2 Problem Statement and Research Aims	2
1.3 Research Questions and Objectives	2
2 LITERATURE REVIEW	4
2.1 Classical Vector Search	4
2.2 Cross-Modal Embeddings	4
2.3 Quantum Similarity Primitives	5
2.4 Hybrid Classical–Quantum Architectures	5
3 RESEARCH METHODOLOGY	7
3.1 System Overview	7
3.2 Methodology Diagram	8
3.3 Dataset and Ground Truth	10
3.4 Embedding Pipeline	11
3.5 Search Engines	13
3.5.1 Classical Baselines	13
3.5.2 Quantum Engines	14
3.6 Evaluation Metrics	17
3.7 Experimental Configuration	18

Use of AI Tools	19
4 RESULTS AND ANALYSIS	21
4.1 Accuracy Across Engines	21
4.2 Quantum Resource Cost	23
4.3 Wall-Clock Behaviour (Within-Engine Only)	25
4.4 IBM Hardware Sanity Check	26
4.5 Discussion	27
4.6 Limitations	28
5 SUSTAINABILITY, INCLUSIVITY, AND SOCIETAL IMPACT	29
6 CONCLUSION	30
REFERENCES	31
A APPENDICES	32
APPENDICES	32
Appendix A: Engine Configuration	32
Appendix B: Database Schema	33
Appendix C: Source Code Repository	33

ABSTRACT

QUANTUM-ENHANCED MULTI-MODAL VECTOR SEARCH: A HYBRID DATABASE WEB APP

Vector search, the task of finding the stored items whose embeddings lie closest to a query, underlies modern retrieval systems, including cross-modal search that retrieves images from a text description. Two quantum primitives, Grover’s algorithm for unstructured search and the swap test for state similarity, appear well suited to this task. This project investigates, through direct measurement, whether they are of practical use for cross-modal vector search at the scale and on the hardware available today.

We built a single framework in which classical, quantum, and hybrid search engines answer the same queries over the same CLIP embeddings, so their rankings can be compared on identical terms. Seven engines were implemented behind a common interface: three classical baselines (an exact scan, an exact FAISS index, and an approximate Hierarchical Navigable Small World graph), three quantum engines (a swap-test estimator and two variants of Grover’s algorithm), and a hybrid that uses the classical index to shortlist candidates and the swap test to rerank them. We benchmarked every engine over a controlled twenty-image cross-modal dataset across a range of embedding dimensions and measurement shot counts, recording retrieval accuracy as Mean Reciprocal Rank alongside hardware-independent cost measures such as oracle calls, qubit count, and circuit depth. The quantum engines ran on a classical simulator, with a single small validation run on real IBM quantum hardware.

The results show that retrieval accuracy was governed by the embedding dimension rather than by whether an engine was quantum. The swap test reproduced the classical ranking accuracy once given a sufficient measurement budget, matching it exactly at the highest dimension, and our Grover implementation reproduced its theoretical oracle-call count exactly. No engine that performed its search quantumly outperformed the classical graph-based baseline, however, and the hybrid engine matched that baseline only because the classical index performed the search. The underlying reason is structural: vector retrieval is dominated by reading the dataset, a data-bound task, whereas quantum advantage concentrates on compute-bound problems over small inputs. We conclude that cross-modal vector search is not a natural fit for quantum methods, even setting aside the maturity of current hardware, and that the apparent

speed-up of quantum search rests on a data-loading cost that the headline figures omit. The contribution of this work is an honest, reproducible, like-for-like comparison and an explicit account of where the quantum-classical gap actually lies.

Keywords: *quantum computing, vector search, similarity search, swap test, Grover's algorithm, CLIP, benchmarking*

LIST OF TABLES

Table 1:	Benchmark configuration grid.	19
Table 2:	Mean Reciprocal Rank by engine and dimension	21
Table 3:	Operations per query and complexity class	23
Table 4:	Quantum circuit width and depth	25
Table 5:	Mean query time by engine and dimension	26

LIST OF FIGURES

Figure 1:	End-to-end system pipeline	8
Figure 2:	Experimental methodology flow	9
Figure 3:	Swap-test circuit	15
Figure 4:	Grover circuit, one iteration	16
Figure 5:	Grover oracle-call scaling	24

1. INTRODUCTION

Modern machine-learning systems represent text, images, and other media as high-dimensional vectors called embeddings, arranged so that items with similar meaning sit close together in the vector space. Searching a collection then reduces to finding the stored vectors nearest to a query vector, a task known as vector search. It underlies image search, recommendation systems, and the retrieval step of retrieval-augmented language models. When the query and the stored items are different kinds of media, such as retrieving images from a written description, the task is cross-modal retrieval, and it becomes tractable once a single embedding model places both modalities in one shared space.

Quantum computing offers two primitives that appear, at first sight, well matched to this task. Grover’s algorithm finds a marked item in an unstructured collection using far fewer queries than a classical scan, and the swap test estimates how similar two quantum states are. This study asks, through measurement rather than assertion, whether those primitives are of practical use for cross-modal vector search at the scale and on the hardware available today. It is an investigation of where quantum search does and does not fit, not an attempt to demonstrate a quantum advantage.

1.1 Background of Study

An embedding model maps an input to a fixed-length vector so that semantically related inputs map to nearby vectors. For cross-modal retrieval we use CLIP, a model trained on a large collection of image and caption pairs to place an image and its description close together in a shared space [1]. Once both a text query and a set of images live in that space, answering the query means ranking the image vectors by their closeness to the query vector. The simplest way to do this is an exhaustive scan that compares the query against every stored vector, which costs $O(N)$ comparisons for N items. Production systems reduce this cost with specialised libraries such as FAISS, which performs the exact scan efficiently [2], or with approximate methods such as the Hierarchical Navigable Small World (HNSW) graph, which reaches a near neighbour in roughly $O(\log N)$ steps by navigating a layered proximity graph [3].

The quantum side of the comparison rests on two ideas. Grover’s algorithm searches an unstructured collection of N items for a marked one using only about $\sqrt{N}$

queries to an oracle, a quadratic improvement over the linear classical scan [4]. The swap test takes two quantum states and estimates the squared overlap between them from a single measured qubit [5]; because a normalised embedding can be encoded in the amplitudes of a quantum state, that overlap is a similarity score. Both techniques therefore have a natural reading as vector-search operations, Grover as the search itself and the swap test as the similarity comparison.

The difficulty is that these speed-ups assume the data is already available inside the quantum computer. For Grover to search real vectors, an oracle would have to recognise the closest item while the whole dataset is held in superposition, which requires a quantum random-access memory that does not yet exist at usable scale; loading the data one item at a time instead costs $O(N)$ and removes the very advantage the algorithm promises [6]. This tension between an algorithm’s asymptotic promise and the cost of getting data into the machine is the central question the study examines.

1.2 Problem Statement and Research Aims

Classical vector search is mature, fast, and well understood. Whether quantum methods add anything to it at the scale available today is far less clear, and popular accounts often overstate the case. The gap this work addresses is empirical: there are few honest, like-for-like comparisons that run classical and quantum similarity search on the same data and report both retrieval accuracy and the resources each method consumes. We built such a comparison. A single common interface lets classical, quantum, and hybrid engines answer the same queries over the same embeddings, so their rankings can be placed side by side and scored identically.

We frame the work as a measurement rather than a proof of advantage. Present-day devices are small and noisy, the regime usually called noisy intermediate-scale quantum (NISQ), so we run the quantum engines mostly on a classical simulator and validate the hardware path with a single small run on a real quantum processor. The goal is to determine, on a controlled cross-modal dataset, how the quantum and hybrid engines compare with strong classical baselines, and to state plainly the conditions under which each approach is preferable.

1.3 Research Questions and Objectives

RQ1. *Accuracy*: Does swap-test similarity reproduce the ranking of classical cosine similarity, and what is the shot budget required to do so?

- RQ2. *Scaling*: Does our Grover implementation reproduce the $\lfloor \pi\sqrt{N}/4 \rfloor$ oracle-call scaling in practice?
- RQ3. *Practicality*: Under what conditions – if any – would a quantum vector-search engine outperform a production classical baseline (HNSW) on the same dataset?
- RQ4. *Suitability*: Independent of hardware maturity, is cross-modal vector search a problem class that quantum computing is suited to, given that quantum advantage concentrates on compute-bound problems over small data rather than data-bound retrieval?

To answer these questions we set four objectives, each corresponding to a concrete artefact in the project repository:

1. Implement a suite of search engines spanning classical, quantum, and hybrid approaches behind a single common interface, so that one query can be answered by any of them and the results compared directly.
2. Build a reproducible benchmark harness and a controlled cross-modal dataset, and collect a complete set of results across embedding dimensions and measurement shot counts.
3. Provide a web application that displays the engines' rankings side by side for inspection.
4. Produce an honest written analysis that answers the research questions from the measured data, including the cases where quantum methods do not help.

2. LITERATURE REVIEW

2.1 Classical Vector Search

The baseline for vector search is an exact scan: the query vector is compared against every stored vector, and the closest are returned. This guarantees the correct nearest neighbours but costs $O(N)$ comparisons per query, which becomes the bottleneck as a collection grows. FAISS is a widely used library that performs this exact search efficiently by exploiting the vectorised instructions of modern processors, and extends it to very large collections [2]. When exact search is too slow, approximate nearest-neighbour methods accept a small loss in recall for a large gain in speed. The most widely used of these is the Hierarchical Navigable Small World (HNSW) graph, which links each vector to a set of near neighbours across several layers and answers a query by greedily descending the graph, reaching a near neighbour in roughly $O(\log N)$ steps [3]. HNSW is the de facto production standard for similarity search, and it is the most demanding classical baseline against which a quantum method must be judged: a quantum search that hopes to be useful has to improve on a classical method that is already sublinear.

2.2 Cross-Modal Embeddings

Cross-modal retrieval depends on embedding different kinds of media into a common space. CLIP (Contrastive Language-Image Pre-training) does this for text and images by training a text encoder and an image encoder together, so that an image and its caption map to nearby vectors while mismatched pairs are pushed apart [1]. The variant we use pairs a Vision Transformer image encoder (ViT-B/32) with a text encoder and produces 512-dimensional embeddings, in which the cosine similarity between a text vector and an image vector measures their relevance. This reduces text-to-image retrieval to ordinary nearest-neighbour search in a single vector space. For the quantum engines, which encode a vector into the amplitudes of a quantum state, we truncate these 512-dimensional embeddings to 64, 128, or 256 components to keep the number of simulated qubits manageable, a trade-off examined in the methodology and results chapters.

2.3 Quantum Similarity Primitives

Two quantum primitives are relevant to similarity search. Grover’s algorithm searches an unstructured collection of N items for one that satisfies a condition checked by an oracle, and it does so with about $\lfloor (\pi/4)\sqrt{N} \rfloor$ oracle queries, a quadratic improvement on the N checks a classical scan requires [4]. The swap test, introduced in work on quantum fingerprinting, compares two quantum states by placing an ancilla qubit in superposition, conditionally swapping the two states, and measuring the ancilla; the probability of reading the ancilla as zero is $(1 + |\langle \psi | \phi \rangle|^2)/2$, so the test recovers the squared overlap of the states and with it a similarity score [5].

Both primitives assume the data is already present as quantum states, and this assumption carries the cost that the asymptotic figures hide. Encoding N vectors into a quantum state, or building an oracle that recognises the closest of them, requires reading all N items. A quantum random-access memory has been proposed to supply data items in superposition [7], but no such device exists at usable scale, and if the data must instead be loaded one item at a time, the $O(N)$ preparation cost cancels Grover’s quadratic advantage at the level of the whole system [6]. The present work therefore treats Grover’s scaling as a property of the search step in isolation and is explicit about the data-loading cost it does not include, a distinction returned to in the results chapter.

2.4 Hybrid Classical–Quantum Architectures

The retrieve-and-rerank cascade is a standard pattern in information retrieval: a fast, inexpensive first stage selects a small set of candidates from the whole collection, and a slower but more accurate second stage reorders only those candidates. The cost of the expensive stage is then bounded by the size of the shortlist rather than by the size of the collection. This pattern is a natural fit for near-term quantum hardware. Quantum machine learning remains constrained by considerable hardware and software challenges [8], so a practical near-term design keeps the global, data-bound work on classical hardware and reserves the quantum processor for a small, structured subproblem.

Our `hybrid_hnsw_swap_test` engine is an instance of this cascade. The classical HNSW index of Section 2.1 selects a small candidate pool in roughly $O(\log N)$ steps, and the swap test of Section 2.3 reranks only those candidates by their estimated similarity, executed on the Qiskit Aer simulator [9]. Because the swap test runs one circuit per candidate, confining it to the shortlist keeps the number of circuit executions independent of the collection size, at $O(\log N + M)$ for a shortlist of M candidates. This

places the quantum step where its cost is bounded and where an estimation error can only reorder an already-relevant shortlist rather than miss a relevant item entirely, an arrangement whose accuracy and cost we evaluate in the results chapter.

3. RESEARCH METHODOLOGY

3.1 System Overview

The system is a modular monolith that performs cross-modal retrieval: given a short text query such as “a dog on a beach”, it returns the dataset images whose content best matches that description. Text and images are never compared directly. Instead, the same embedding model maps both into a shared vector space, and retrieval becomes the problem of finding the dataset vectors closest to the query vector. The defining property of the system is that the component performing this closest-vector search is interchangeable. A classical engine, a quantum engine, and a hybrid of the two all expose one common interface, so a single query can be answered by any of them and the rankings placed side by side. This makes the system an instrument for comparing search strategies rather than a single retrieval product.¹

This interchangeability is enforced by a single design decision that runs through the whole codebase: the strategy pattern. Each stage of the pipeline is defined by an abstract interface, and concrete implementations plug into it. Data loaders implement `BaseDataLoader`, embedding models implement `EmbeddingGenerator`, and every search engine implements `SearchEngineStrategy`, which requires just two operations, `build_index` and `search`. Because all engines satisfy the same contract, the surrounding harness that loads data, generates embeddings, runs queries, and records results is identical for every engine. Any difference we measure between a classical and a quantum engine therefore reflects the engine itself, not the code around it. Which engines run, at which vector dimensions, and with how many measurement shots is selected in a configuration file (`benchmarks.yaml`) rather than by editing source, so an experiment is defined by configuration rather than by code changes.

The pipeline is organised into five phases, shown in Figure 1. In the first phase, data loading, a `DirectoryDataLoader` reads the image files and the accompanying ground-truth pairs that define which image each query should retrieve. In the second phase, the embedding pipeline, a CLIP model converts every image and every text query into a vector. In the third phase the search engines build their indexes over these vectors and answer queries, each engine applying its own strategy, whether classical, quantum,

¹The full source code, benchmark harness, and dataset configuration are publicly available at <https://github.com/Mirza404/quantum-vector-search>.

or hybrid. In the fourth phase the outcome of every run, including its accuracy and resource measurements, is written to a PostgreSQL database. The fifth phase exposes the system through a FastAPI backend and a React interface, which lets a user issue a query and inspect the rankings returned by each engine. The sections that follow describe each phase in turn: the dataset and ground truth in Section 3.3, the embedding pipeline in Section 3.4, and the individual search engines in Section 3.5.

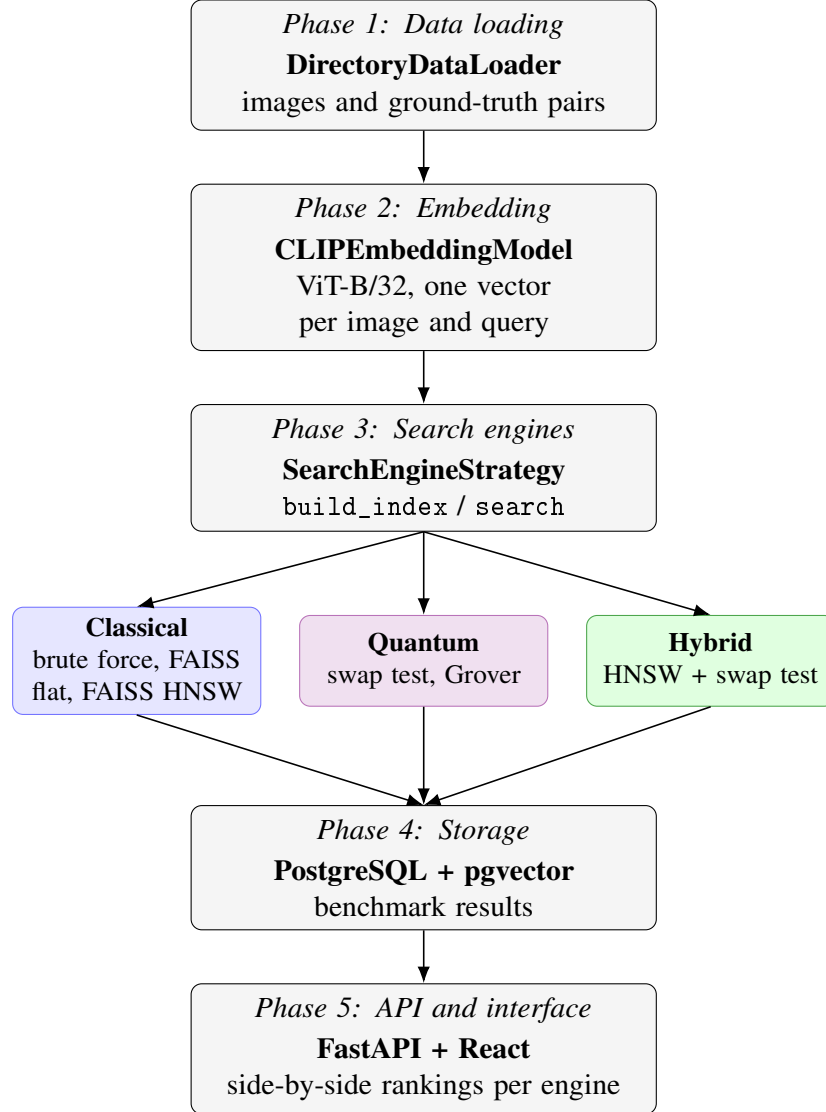


Figure 1: End-to-end pipeline. Data loading and CLIP embedding feed a common engine-strategy interface, behind which the classical, quantum, and hybrid engines are interchangeable. Every run is recorded in PostgreSQL and surfaced through the FastAPI and React interface.

3.2 Methodology Diagram

The study follows a fixed sequence of steps, shown in Figure 2, each implemented by a single script so the whole experiment can be reproduced from a clean checkout. The

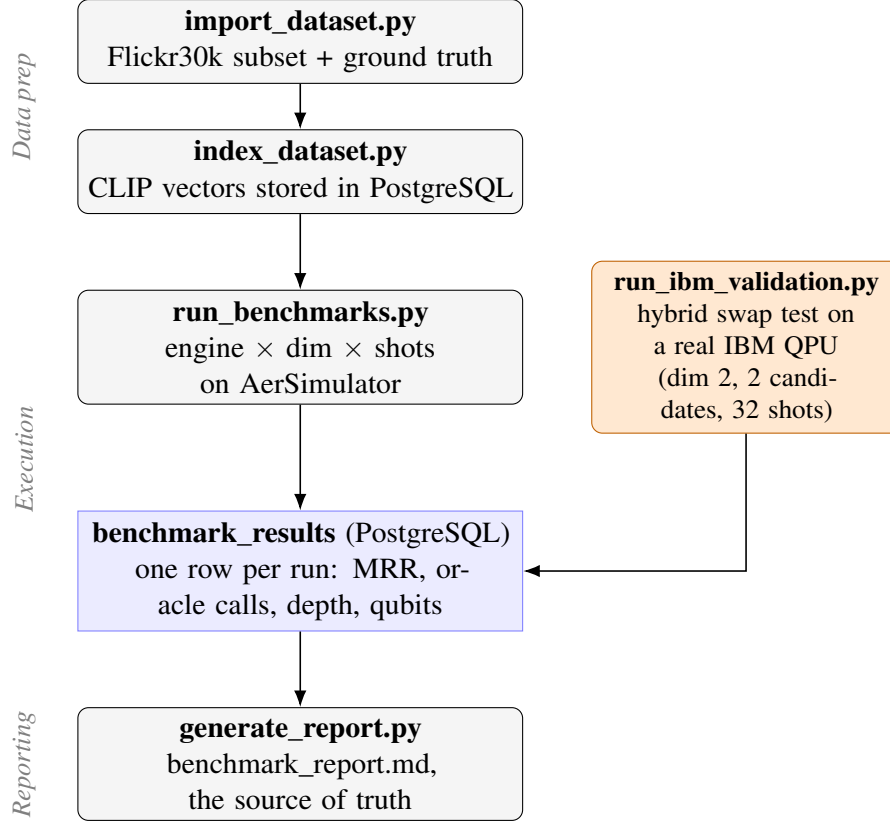


Figure 2: Methodology flow. Four scripts run in order, from dataset import through CLIP indexing, the simulator benchmark sweep, and report generation. The IBM hardware validation is a one-off side arm that writes its single result into the same database table.

first two steps prepare the data. The script `import_dataset.py` downloads a fixed subset of the Flickr30k dataset and, in the same pass, writes the ground-truth file that records which image each text query should retrieve. This download is deterministic: every run regenerates the same image set and the same query-to-image pairing, so the results do not drift between machines. The second script, `index_dataset.py`, loads each image, passes it through the CLIP model to obtain a vector, and stores that vector in PostgreSQL. After these two steps the database holds one embedding per image, ready to be searched.

With the embeddings in place, `run_benchmarks.py` carries out the main experiment. It iterates over every combination of engine, vector dimension, and shot count listed in `benchmarks.yaml`, and for each combination it answers all of the ground-truth queries. The quantum and hybrid engines execute their circuits on Qiskit’s `AerSimulator`, a noiseless simulator that isolates the effect of finite sampling (shot noise) from the physical noise of real hardware. Every individual run is recorded as one row in the `benchmark_results` table, together with its accuracy and its resource cost: the Mean Reciprocal Rank of the retrieved images, and, for the quantum engines, the

number of oracle calls, the circuit depth, and the qubit count. Because the configuration file drives the sweep, widening the experiment to a new dimension or shot count is a change to that file alone, not to the code.

Alongside the simulator sweep, `run_ibm_validation.py` runs the hybrid swap-test engine once on real IBM quantum hardware, at the smallest workable size: two dimensions, two candidate images, and 32 shots. Its purpose is narrow, to confirm that the swap-test circuit executes on a physical device and returns a sensible result, rather than to measure scaling. The single outcome is written to the same `benchmark_results` table, tagged as the `hybrid_hnsw_swap_test_ibm` engine, so it sits beside the simulator runs without being confused for one.

The final step turns the stored runs back into something readable. The script `generate_report.py` queries the `benchmark_results` table and produces a Mark-down summary, `benchmark_report.md`, that groups the runs by engine and reports accuracy per dimension and per shot count. This report is the single source of truth for the Results chapter: every figure quoted in Chapter 4 traces back to a row this script read from the database, rather than to any number recorded by hand. Keeping the analysis in one generated file, derived directly from the stored runs, is what lets the whole study be regenerated by re-running the four scripts in order.

3.3 Dataset and Ground Truth

The evaluation set is a fixed 20-image subset of Flickr30k, a widely used corpus of everyday photographs in which each image is paired with several short captions written by human annotators. We chose an image-caption corpus because it gives us, at no extra labelling effort, a natural pairing between a text query and the image it describes, which is exactly the cross-modal retrieval task our system performs. The subset is assembled deterministically: the importer requests a fixed dataset configuration and split and reads the first twenty rows in the order the dataset viewer returns them, with no random sampling and no shuffling. Running the import on a different machine therefore reproduces the same twenty images and the same queries, which is what allows the benchmark to be regenerated from scratch rather than depending on a particular download. We are explicit that twenty images is a small set by the standards of information-retrieval research. This is a deliberate consequence of the study’s goal: a controlled comparison of the seven active benchmark engines, several of which run on a quantum-circuit simulator on a CPU, where the cost of each query grows quickly with the size of the candidate pool. The separate IBM hardware validation is intentionally excluded from this count because it is a tiny real-device sanity check rather than part

of the normal simulator sweep. The subset is large enough to separate the engines by accuracy while keeping the quantum simulations tractable, and the importer exposes the count as a single configuration value so the set can be widened when more compute is available.

Each Flickr30k image carries several captions, since multiple annotators described the same photograph independently. For each image we take the first caption as the query text and discard the rest, so every query is a real sentence a person wrote about a specific image. This gives the evaluation a one-to-one structure: each of the twenty queries has exactly one correct target, namely the image its caption was written for. The importer records this pairing in a small generated file, keying each query to its image by the image’s original Flickr identifier so that the link between a query and its target is stable across re-imports and never depends on load order. Discarding the alternate captions keeps the task unambiguous, at the cost of ignoring the fact that a second annotator might have phrased the same query differently. We accept that trade-off because it makes “correct” a single, checkable fact rather than a matter of degree.

Because the pairing comes from the dataset itself, the ground truth is intrinsic: we never collect separate human judgements of which images are relevant to which queries. An engine answers a query by ranking all twenty images, and we read off the position of the one image the caption was written for. A retrieval is counted as fully correct when that image is ranked first; when it appears lower down, the engine receives partial credit in inverse proportion to its rank, which is the mechanism behind the mean reciprocal rank reported in Chapter 4. Defining correctness this way removes all subjectivity from scoring, since there is exactly one right answer per query and no annotator has to decide how relevant a near-miss is. The cost is that the metric cannot reward an engine for surfacing a different image that a human might also have judged a reasonable match, because the dataset does not record any such alternative. For a study whose aim is to compare engines against one another under identical conditions, rather than to estimate absolute retrieval quality for users, this strict and reproducible notion of correctness is the appropriate choice.

3.4 Embedding Pipeline

Both the image collection and the incoming query are turned into vectors by CLIP, a vision-language model that learns a single embedding space shared between pictures and text [1]. CLIP is trained contrastively on a large collection of image-caption pairs gathered from the web, with the objective of placing an image and its matching caption

close together while pushing mismatched pairs apart [1]. The practical consequence for our system is that a sentence and the photograph it describes land near one another in the same space, which reduces text-to-image retrieval to finding the image vectors nearest the query vector. We use the ViT-B/32 variant, which represents each input as a 512-dimensional vector, and run it on whatever accelerator is available, falling back to the CPU when there is no GPU so that the pipeline is reproducible on a plain machine.

Before any vector leaves the embedding stage, we L2-normalise it, scaling both the image vectors and the query vector to unit length. This is not a cosmetic step: once every vector lies on the unit sphere, cosine similarity, Euclidean distance, and the squared inner product that the swap test estimates all become monotonic transforms of one another. They report different numbers, but they order the candidate images identically. That equivalence is what makes a fair comparison possible at all, because it lets us hold the ranking task fixed while varying only the engine that computes it, and it is why the mean reciprocal rank defined in Chapter 4 can be read directly across engines that nominally optimise different quantities.

The embeddings are not used at their full width of 512 dimensions. Instead, we truncate each vector to a smaller size by keeping its leading components, and we sweep several such sizes, 64, 128, and 256, across the benchmark. The reason is the cost of simulating the quantum engines on a classical processor. To load a d -dimensional vector into a quantum register by amplitude encoding requires $\lceil \log_2 d \rceil$ qubits, and the swap test compares two such registers through one extra control qubit, for a total of $1 + 2\lceil \log_2 d \rceil$ qubits. The simulator must track a complex amplitude for every possible state of those qubits, so its memory and time grow as two raised to that count. At the full 512 dimensions the swap test would need nineteen qubits, and therefore a state vector of more than half a million amplitudes per comparison; truncating to 64 dimensions brings this down to thirteen qubits and roughly eight thousand amplitudes, which is what keeps the full sweep feasible on an ordinary CPU. Truncation is not free: discarding components costs accuracy, an effect the Results chapter measures directly, and because normalisation was applied to the full vector the truncated vector is no longer of unit length, so the engines that depend on that property re-normalise it before use.

3.5 Search Engines

3.5.1 Classical Baselines

Brute-Force Cosine

The simplest engine, and the one we treat as the reference, compares the query against every image in turn. Each stored vector and the query are normalised to unit length, after which the cosine similarity between two vectors is just their dot product; the engine computes this product for all twenty images, sorts the scores, and returns the highest. Because it inspects every candidate, the work grows linearly with the size of the collection, written $O(N)$, and it can never overlook the true closest image. That exhaustiveness is precisely why we adopt its output as the ground-truth ranking: every other engine in the study, classical or quantum, is judged by how closely its ordering reproduces the one brute-force search produces. The engine therefore plays two roles at once, a search method we time like any other and the yardstick against which the accuracy of all the others is measured.

FAISS Flat (Exact)

The second classical engine delegates the same exhaustive search to FAISS, an open-source similarity-search library whose underlying methods cover exact, approximate, and compressed-domain search [2]. We use its exact index, which scores every candidate but measures Euclidean (L2) distance rather than cosine similarity and evaluates many distances at once using the vector instructions of a modern CPU. On our data the two engines return the same ordering, because for unit-length vectors L2 distance and cosine similarity are tied by the identity $\|a - b\|^2 = 2 - 2\cos\theta$: a smaller distance always means a larger cosine. That equivalence holds only because we re-normalise the vectors after truncation, since truncating the CLIP output to a lower dimension breaks the unit norm; without that step the exact L2 ranking would drift away from the cosine ground truth. FAISS Flat thus serves as a faster, production-grade stand-in for brute force that should, and does, reproduce the reference ranking exactly.

FAISS HNSW (Approximate)

The third classical engine trades exactness for speed. Rather than scan every image, it queries a Hierarchical Navigable Small World (HNSW) graph, an approximate-nearest-neighbour index that organises the vectors into a multi-layer structure of proximity graphs and descends from a sparse top layer to the dense bottom one, following edges toward the query at each level [3]. Because the search visits only a small neighbourhood

of the graph rather than the whole collection, its cost scales logarithmically, $O(\log N)$, which is what makes HNSW the standard choice for large production retrieval systems. The trade-off is that it can occasionally miss the true nearest image, though in practice its recall is very high. We include it through the same FAISS library, configured with its default graph parameters, and re-normalise the vectors exactly as for the exact index. On our 20-image collection the graph is far too small for the approximation to cost anything, so HNSW is expected to, and does, reproduce the exact ranking; its value here is to place the quantum engines next to the algorithm a real system would actually deploy, not merely next to brute force.

3.5.2 Quantum Engines

Qiskit Swap Test

The swap test estimates the squared inner product of two amplitude-encoded vectors $|\psi\rangle$ and $|\phi\rangle$ via a single ancilla measurement:

$$P(\text{ancilla} = 0) = \frac{1 + |\langle\psi|\phi\rangle|^2}{2}. \quad (1)$$

The procedure originates in work on quantum fingerprinting, which introduced a test able to distinguish two unknown quantum states with high probability [5]. In our engine each vector is amplitude-encoded into its own quantum register, and a single ancilla qubit acts as a referee. The circuit applies a Hadamard gate to the ancilla, performs a controlled swap of the two registers conditioned on it, applies a second Hadamard, and measures the ancilla; Figure 3 shows this circuit for a four-dimensional example. Equation 1 relates the probability of reading the ancilla as zero to the overlap of the two states, so by estimating that probability and inverting the relation we recover a similarity score. The estimate is never exact, because the probability is read off a finite number of repeated measurements, called shots; the statistical error shrinks only as one over the square root of the shot count, which is why we sweep the number of shots and study how quickly the accuracy approaches the classical ceiling. Crucially, the swap test compares the query against one candidate at a time, so ranking the whole collection still requires one circuit per image, $O(N)$ circuits in total. It is therefore a similarity estimator rather than a search algorithm, and it offers no asymptotic speedup over brute force. We run every circuit on Qiskit’s Aer statevector simulator on a CPU rather than on real hardware [9], because the sweep involves thousands of executions across several dimensions and shot counts; a separate, deliberately small run on an IBM quantum processor, described below, checks that the simulated behaviour matches a

physical device.

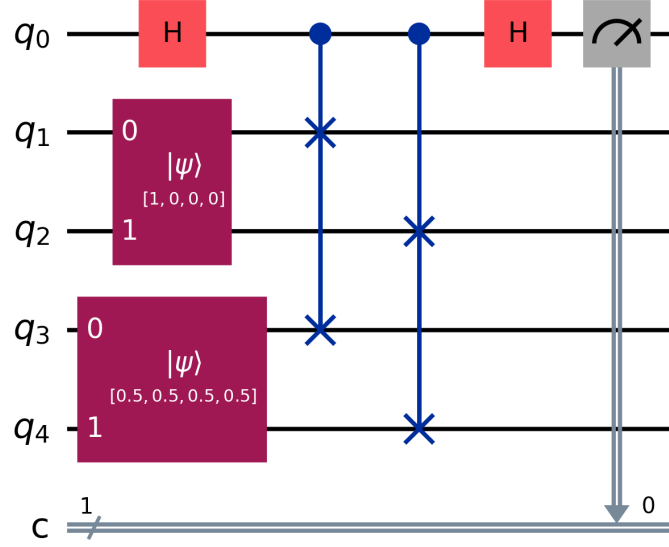


Figure 3: Swap-test circuit for a four-dimensional example, in which each vector is amplitude-encoded into two qubits. The ancilla (q_0) is placed in superposition, controls a swap of the two encoded registers (q_1q_2 and q_3q_4), and is measured after a second Hadamard. The probability of reading the ancilla as zero gives the squared overlap of the two states, as in Equation 1.

Qiskit Grover (Hardcoded Oracle)

Grover’s algorithm searches an unstructured collection of N items for a marked one using only about the square root of N queries to an oracle, a quadratic improvement over the N comparisons a classical scan needs [4]. Our engine implements the textbook circuit: it places an index register of $\log_2 N$ qubits, with N padded up to a power of two, into uniform superposition, then repeats an oracle followed by a diffusion operator $\lfloor \pi\sqrt{N}/4 \rfloor$ times before measuring; Figure 4 shows one such iteration for a four-element index. The oracle flips the phase of the target index and the diffusion step amplifies its amplitude, so the measurement returns the marked index with high probability. The honest difficulty is the oracle itself. For this to be a genuine search, the oracle would have to compute, in superposition, which of the N stored vectors is closest to the query, which in turn requires loading all N vectors into the quantum state through a quantum random access memory, a device that addresses a superposition of memory cells [7]. No such hardware exists at usable scale, and if instead the data has to be loaded one item at a time the preparation alone costs $O(N)$, which erases the quadratic advantage before the search even begins [6]. We therefore make the limitation explicit rather than hide it: the engine finds the nearest neighbour classically and constructs an oracle that marks

exactly that index. This isolates the search step so that we can measure its oracle-call count and confirm that it follows the $\lfloor \pi\sqrt{N}/4 \rfloor$ curve, which is the scaling claim we are actually testing, not an end-to-end speedup we are claiming to have achieved.

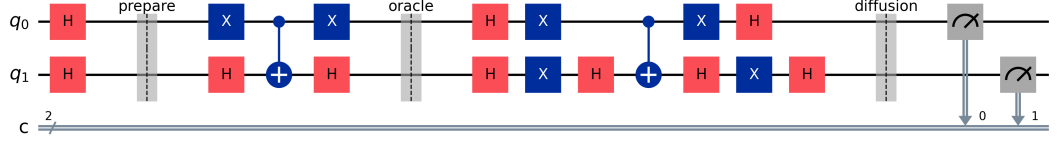


Figure 4: One Grover iteration for a four-element index ($N = 4$, two index qubits). After preparing a uniform superposition, the oracle flips the phase of the marked index and the diffusion operator amplifies its amplitude; the two qubits are then measured. The barriers separate the preparation, oracle, and diffusion phases.

Qiskit Grover with Quantum State Preparation

The final Grover variant removes the one classical shortcut that the previous engine kept. Instead of locating the nearest neighbour with a classical dot product, it determines the target index quantum-mechanically: each stored vector and the query are loaded with Qiskit state-preparation gates, a swap test estimates their overlap, and the index with the highest measured overlap becomes the one the Grover oracle marks. Everything after that, the superposition, the oracle and diffusion iterations, and the final measurement, is identical to the hardcoded variant. We include it to see what happens when the target selection is itself made quantum, and therefore noisy, rather than exact. The outcome, previewed here and quantified in the Results chapter, is that accuracy falls: state preparation produces deep circuits, the per-candidate swap test carries the same shot noise as the standalone swap-test engine, and any error in the quantum target selection is then faithfully amplified by Grover, which has no mechanism to recover once it has been pointed at the wrong index. This engine is thus the most fully quantum of those we study and also the least accurate, a tension that is itself one of the findings the project set out to document.

Hybrid HNSW + Swap Test

The hybrid engine is the one design in the study that tries to place the quantum component where it might plausibly help rather than use it as a standalone search. It runs in two stages. First the classical HNSW index retrieves a small pool of candidates, by default the five nearest, cheaply narrowing the twenty images down to a handful. The swap-test reranker then runs only on that pool, scoring each surviving candidate against the query and reordering them. Because the expensive quantum step is applied to a fixed-size shortlist instead of the whole collection, the number of circuits no longer

grows with the dataset: the classical stage absorbs the $O(N)$ work and the quantum stage does a constant amount of refinement on top. This mirrors the way approximate and exact stages are already combined in production retrieval, where a fast approximate index proposes candidates and a slower, more precise scorer reranks them, and it is the most defensible place to insert a quantum estimator given that the swap test offers no search speedup of its own. On our small collection the shortlist almost always already contains the correct image, so the hybrid tracks the classical baseline closely; its value here is architectural, showing the shape a useful quantum contribution would take rather than a raw accuracy gain.

IBM Hardware Validation

To check that the quantum component is not merely a simulator artefact, we ran the hybrid engine once on a real IBM quantum processor through the IBM Quantum runtime, with the swap-test reranking executed on physical hardware rather than the Aer simulator. This run is deliberately tiny. Vectors are truncated to just two dimensions, the candidate pool is two images, and each circuit is measured with thirty-two shots, a configuration small enough to fit within the limited free-tier processor time and shallow enough that a noisy device can execute it at all. It is a sanity check, not a scaling experiment: the aim is to confirm that the encode, swap-test, and measure pipeline behaves on real hardware as it does in simulation, and to expose honestly how much device noise and aggressive truncation cost in accuracy. The resulting quality, reported with the other measurements in the Results chapter, is far below the simulated engines and close to chance, which is the expected consequence of two-dimensional vectors and thirty-two shots on a noisy processor rather than a failure of the method. We draw no scaling conclusions from it; its role is to ground the rest of the study, which runs on a simulator, against at least one execution on a physical quantum computer.

3.6 Evaluation Metrics

The primary measure of accuracy is the mean reciprocal rank (MRR), a standard information-retrieval metric. For each query it rewards the engine with the reciprocal of the rank at which the correct image appears: a correct image ranked first scores one, ranked second one half, ranked fifth one fifth, and a correct image that falls outside the top k results returned, with k fixed at ten throughout, scores zero. Averaging this reciprocal rank over all queries gives the MRR reported for an engine, as defined in Equation 2. Because every engine is scored against the same ground-truth ranking and, as established in Section 3.4, the underlying similarity measures induce identical orderings on unit vectors, a single MRR value can be compared directly from one

engine to the next.

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}, \quad (2)$$

where rank_q is the position of the first relevant result for query $q \in Q$ and $1/\text{rank}_q := 0$ if no relevant result is in the top- k .

Accuracy alone does not capture why a quantum approach might ever be worth the trouble, so we also record how each engine scales. The natural measure is the operation count, the number of times an engine runs its core operation for a single query, which is independent of the machine it runs on: a slow laptop and a fast server execute the same number of iterations. The exhaustive engines, brute force, FAISS flat, and the standalone swap test, perform one comparison or one circuit per image and so scale as $O(N)$; HNSW traverses its graph in $O(\log N)$; the hybrid does an $O(\log N)$ prefilter followed by a fixed number M of swap tests, $O(\log N + M)$; and Grover completes its search in $\lfloor \pi\sqrt{N}/4 \rfloor$ oracle cycles, $O(\sqrt{N})$. We store this figure in the oracle-calls field of each result, and it is the only speed-like quantity we compare across engines.

For the quantum engines we additionally record two circuit properties that determine whether a circuit could run on real hardware. Circuit depth counts the sequential layers of gates and is the main proxy for how much noise a physical device would accumulate, since today’s machines execute only a limited number of layers reliably. Qubit count is the width of the circuit, which for the swap test is the $1 + 2\lceil \log_2 d \rceil$ derived earlier. These fields are populated only by the engines that build circuits; the classical engines leave them empty. We deliberately do not compare wall-clock time across engines. The quantum engines run on a simulator that reproduces quantum behaviour by manipulating an exponentially large state vector on an ordinary CPU, so their runtimes mostly measure the cost of that simulation rather than anything intrinsic to the algorithm; comparing them with classical CPU timings would penalise the quantum methods for the wrong reason. We therefore report wall-clock only within a single engine, where it is meaningful, and never as a cross-engine ranking.

3.7 Experimental Configuration

Table 1 lists the parameter grid the benchmark sweeps. The harness runs the full cartesian product of these values: every engine is evaluated at each vector dimension, and the four quantum engines, which accept a shot count, are additionally run at each shot setting, while the three classical engines ignore shots and are run once per

dimension. Each resulting configuration is evaluated on all twenty queries. This produces fifteen engine-and-shot configurations per dimension, three classical and four quantum at three shot counts each, forty-five across the three dimensions, and nine hundred individual query runs once multiplied by the twenty queries. The separate IBM hardware validation contributes a further twenty runs, one per query, for a stored total of nine hundred and twenty results. The layers parameter is held at one throughout; the harness requires it but no engine enabled here uses it. Every run is written to a PostgreSQL table together with its full parameter set, and the database is dumped to a seed file committed alongside the code, so the entire dataset can be reconstructed exactly rather than trusted as a one-off snapshot.

Table 1: Benchmark configuration grid.

Parameter	Values
Engines	Brute-force cosine, FAISS flat, FAISS HNSW (classical); swap test, Grover (hard-coded oracle), Grover (quantum prep), hybrid HNSW + swap test (quantum)
Dimensions	64, 128, 256
Shots	512, 1024, 2048 (quantum engines only)
Layers	1
Top- k	10

Use of AI Tools

This project made use of AI assistants, specifically Claude Code and ChatGPT, as productivity tools during development and writing. Their role was to accelerate well-defined, labour-intensive tasks under the authors’ direction, not to originate the work. Concretely, the tools assisted with generating and refactoring routine code, reviewing changes and helping locate bugs during the code audit, drafting prose that the authors then revised, and rendering diagrams from layouts the authors specified.

The intellectual core of the project is the authors’ own. The system architecture, including the strategy-pattern engine interface and the five-phase pipeline, the selection and configuration of the search engines, the design of the benchmark and its research questions, and the interpretation of the results were reasoned through and decided by the authors. Every benchmark figure reported here was produced by our own experimental runs, and every external reference was verified by the authors against its primary source before it was cited.

The authors take full responsibility for the correctness, originality, and verifica-

tion of all code, data, and conclusions in this report. Where AI tools contributed text or code, that output was read, checked, and where necessary corrected and rewritten by the authors, who treat it as a draft to be scrutinised rather than as an authority to be trusted.

4. RESULTS AND ANALYSIS

The central result of this study is that retrieval accuracy was governed by the embedding dimension we gave each engine, not by whether the engine was classical or quantum. The three classical baselines returned identical rankings and tied for the highest Mean Reciprocal Rank (MRR) at 0.829, and every engine, classical or quantum, improved along the same staircase as the dimension rose from 64 to 256. Within that pattern the quantum engines behaved exactly as their algorithms predict. The swap test reproduced the classical ranking exactly at dimension 256 (MRR 0.975) and degraded gracefully below it, recovering as we spent more measurement shots. Grover with a hardcoded oracle returned the correct neighbour using a number of oracle calls that matched the theoretical $\lfloor (\pi/4)\sqrt{N} \rfloor$ count, but its sampling-based read-out left it slightly short of the classical ceiling (MRR 0.955 at dimension 256). The fully quantum state-preparation variant of Grover was the weakest engine in the study (MRR 0.559). The hybrid engine, which uses the classical index to shortlist candidates and the swap test only to rerank them, effectively tied the classical baseline (MRR 0.823). The sections below break these findings down by accuracy, quantum resource cost, timing, and the separate hardware validation run, and then return to the research questions.

4.1 Accuracy Across Engines

Table 2: Average Mean Reciprocal Rank (MRR) by engine and embedding dimension, over the twenty benchmark queries. Higher is better: an MRR of 1.000 means the correct image was ranked first for every query. Quantum cells average over the three shot counts (512, 1024, 2048); the shot dependence is examined separately below. The IBM hardware run is reported in Section 4.4.

Engine	MRR by dimension			
	64	128	256	Avg
Brute-force cosine	0.655	0.858	0.975	0.829
FAISS flat	0.655	0.858	0.975	0.829
FAISS HNSW	0.655	0.858	0.975	0.829
Swap test	0.470	0.738	0.975	0.728
Grover (hardcoded oracle)	0.546	0.776	0.955	0.759
Grover (quantum state prep)	0.404	0.551	0.722	0.559
Hybrid (HNSW + swap test)	0.635	0.858	0.975	0.823

The three classical engines are statistically indistinguishable in Table 2: brute-force cosine, FAISS flat, and FAISS HNSW each scored an average MRR of 0.829 and, query by query, produced the same ordering of results. This is the expected outcome rather than a coincidence. Brute-force cosine and FAISS flat are both exact searches over the same twenty unit vectors, so they must agree, and FAISS HNSW is an approximate graph index whose approximation only begins to matter at thousands of vectors. With a dataset this small its navigable graph visits effectively every node, so it reproduces the exact ranking. We treat the brute-force result as the ground-truth ordering against which the other engines are measured.

The dominant factor in accuracy was the embedding dimension. Every engine in the table improves monotonically from dimension 64 to 256, and the classical baseline traces a clear staircase of 0.655, 0.858, and 0.975. This is a property of the embeddings, not of the search algorithm: truncating the 512-dimensional CLIP vectors to 64 components discards information that separates near-neighbours, and restoring dimensions recovers it. Because we truncate for the simulator’s qubit budget rather than for any retrieval reason, the dimension axis is best read as the cost of making the problem small enough to simulate, and the 256-dimensional column as the most representative of what the embeddings can do.

The swap test illustrates how a quantum estimator approaches the exact answer as its measurement budget grows. At dimension 256 it matched the classical ceiling exactly (MRR 0.975 at every shot count), because the overlaps it estimates are well separated and even a coarse estimate ranks them correctly. At dimension 128, where neighbours are closer together, accuracy depended on how finely we measured: MRR rose from 0.642 at 512 shots to 0.688 at 1024 and 0.883 at 2048. The swap test estimates the squared overlap $|\langle \psi | \phi \rangle|^2$ from a finite number of measurements, so additional shots reduce the sampling noise and sharpen the ranking. This shot-versus-accuracy trade-off, rather than any ceiling on the method itself, explains the swap test’s lower average of 0.728: the average is pulled down by the low-dimension, low-shot configurations that a practical deployment would not choose.

The two Grover engines separate sharply. Grover with a hardcoded oracle tracked the classical accuracy closely, reaching MRR 0.955 at dimension 256, just short of the 0.975 ceiling. The shortfall comes from the read-out: Grover amplifies the marked state but is still measured probabilistically, so on a small number of queries the most-sampled index is not the true nearest neighbour. The fully quantum variant, which prepares the candidate state on the circuit instead of supplying it through a classical oracle, was the weakest engine at every dimension and averaged 0.559. Moving the

one classically assisted step into the quantum circuit compounded the state-preparation error and the same sampling noise, and the engine had no classical fallback to recover from a misdirected search. The hybrid engine sits at the opposite end of this trade-off: by letting the classical HNSW index choose the candidates and using the swap test only to rerank them, it stayed level with the classical baseline (MRR 0.823, and 0.975 at dimension 256), confining the quantum step to the part of the pipeline where its error did not change the outcome.

4.2 Quantum Resource Cost

Because the quantum engines run on a classical simulator, their wall-clock time measures simulation overhead rather than the cost of a real quantum device (Section 4.3 returns to this). The hardware-independent way to compare the engines is therefore to count the operations each one performs per query and the resources its circuit would demand on a real device. Table 3 reports the first of these: the number of similarity comparisons or oracle calls each engine makes for a single query over the twenty-image dataset.

Table 3: Operations per query and algorithmic complexity for each engine over the twenty-image dataset ($N = 20$). The operation count is the cross-engine scaling metric; it is independent of hardware. Here M denotes the number of candidates the hybrid engine reranks.

Engine	Operations / query	Complexity
Brute-force cosine	20	$O(N)$
FAISS flat	20	$O(N)$
FAISS HNSW	5	$O(\log N)$ approx.
Swap test	20	$O(N)$
Grover (hardcoded oracle)	4	$O(\sqrt{N})$
Grover (quantum state prep)	4	$O(\sqrt{N})$
Hybrid (HNSW + swap test)	10	$O(\log N + M)$

Grover was the only engine whose per-query cost was sublinear in the dataset size. The exact classical engines and the standalone swap test each performed one comparison per stored vector, twenty in total, while FAISS HNSW reached its answer in about five graph hops and the hybrid engine made ten comparisons (one HNSW shortlist followed by a swap-test rerank of the survivors). Both Grover engines used four oracle calls, and this count is exactly what the theory predicts once the index padding is accounted for. Grover addresses an index register whose length is a power of two, so the twenty images are padded to the next power of two, $N' = 32$, which is why the Grover circuits used five qubits ($\lceil \log_2 32 \rceil = 5$). The optimal number of Grover iterations for

an unstructured search of N' items is $\lfloor (\pi/4)\sqrt{N'} \rfloor$, and for $N' = 32$ this evaluates to $\lfloor 4.44 \rfloor = 4$, matching the measured count. Figure 5 places this verified point on the theoretical curve.

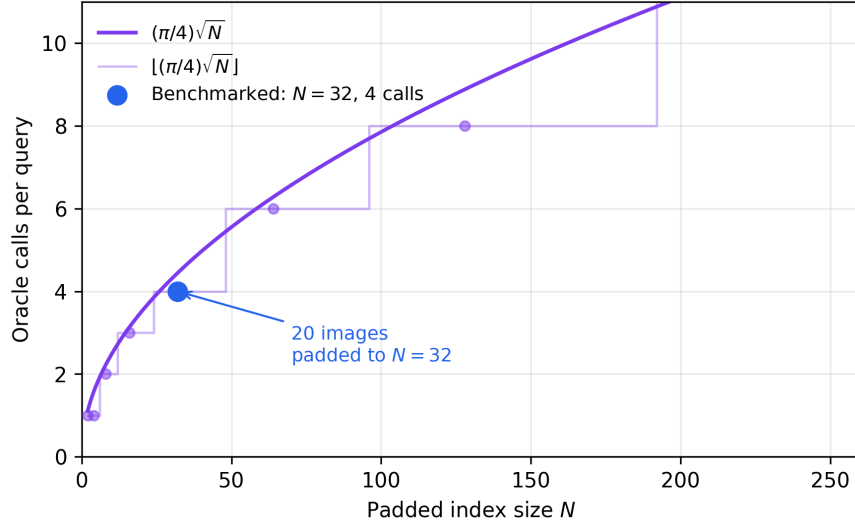


Figure 5: Theoretical Grover oracle-call count $\lfloor (\pi/4)\sqrt{N} \rfloor$ as a function of the padded index size N , with a marker at the single size we benchmarked ($N = 32$, four oracle calls). We ran one dataset size, so this confirms the count at a single point on the curve rather than tracing the curve empirically.

The two quantum families differ sharply in what they encode, and Table 4 shows the consequence for circuit width and depth. Grover encodes the candidate *index*, so its width stayed fixed at five qubits regardless of the embedding dimension; its cost instead appears as depth, around fifty gate layers, because each of the four iterations applies an oracle and a diffusion operator in sequence. The swap test and the hybrid engine encode the candidate *vectors*, so their width grew with the embedding dimension as $1 + 2\lceil \log_2 d \rceil$ qubits, from thirteen at dimension 64 to seventeen at dimension 256, while their circuits stayed shallow (depth nine to eleven). On near-term hardware both costs matter: more qubits are harder to allocate, and greater depth gives decoherence more time to corrupt the state, which is the practical reason the deep Grover circuits would be the harder of the two to run faithfully on a real device.

These counts also expose the assumption hidden inside Grover’s sublinear figure. The four oracle calls measure only the search once the data is already encoded in the oracle; they do not include the cost of building that oracle. Our hardcoded oracle computes the nearest neighbour classically and then marks it, so the twenty comparisons Grover appears to save have in effect been paid before the circuit runs. This is the qRAM caveat set out in Section 3.5: a genuine quantum speed-up for search would require loading the dataset into a quantum-accessible memory, which is itself an

Table 4: Circuit width and depth for the quantum engines, from the compiled circuits. The swap test and the hybrid engine share the same circuit metrics because the hybrid reuses the swap-test circuit. Grover’s width is set by the padded index register, not the embedding dimension, so it is constant.

Engine (dimension)	Qubits	Circuit depth
Swap test / Hybrid, dim 64	13	9
Swap test / Hybrid, dim 128	15	10
Swap test / Hybrid, dim 256	17	11
Grover (both variants), any dim	5	≈ 50

unsolved cost. The swap test and hybrid engines make the opposite trade, paying their cost openly in qubits and in one circuit execution per candidate. Section 4.5 returns to what this means for the practicality question.

4.3 Wall-Clock Behaviour (Within-Engine Only)

Wall-clock time cannot be compared across engines in this study, and Table 5 should be read only one row at a time. The reason is that the quantum engines did not run on a quantum computer: they ran on a classical statevector simulator, which represents the full quantum state in ordinary memory and so does work that grows exponentially with the number of qubits. The times for the quantum engines therefore measure the cost of *simulating* a circuit on a CPU, not the latency a real quantum processor would incur, and putting them next to the classical millisecond figures would compare two unrelated quantities. The within-engine comparison across dimension is meaningful, because there the hardware is held fixed and only the problem size changes.

Read row by row, the table reflects the circuit sizes reported in Section 4.2. The swap test and the hybrid engine slowed sharply as the dimension rose, the swap test from about one second per query at dimension 64 to roughly twenty-two seconds at dimension 256, because each doubling of the embedding dimension adds two qubits to the amplitude encoding and so quadruples the size of the statevector the simulator must track. Grover with a hardcoded oracle was the exception: its time stayed essentially flat at about twenty milliseconds across all three dimensions, because its circuit always uses the same five-qubit index register regardless of the embedding dimension. This contrast is the clearest evidence in our data that the simulation cost is driven by the qubit count, not by the dataset or the dimension as such. The quantum state-preparation variant of Grover was far slower than the hardcoded one, in the thousands of milliseconds, because it builds the candidate state on the circuit instead of supplying it classically.

Table 5: Mean total time per query, in milliseconds, by engine and embedding dimension. The three classical engines completed in well under a millisecond (0.2 to 0.6 ms) and are shown as “< 1”. The quantum and hybrid times are simulator times on a CPU and are *not* comparable to the classical figures or to real quantum hardware; they are reported only to show how each engine’s simulation cost scales with dimension.

Engine	Mean time per query (ms)		
	dim 64	dim 128	dim 256
Brute-force cosine	< 1	< 1	< 1
FAISS flat	< 1	< 1	< 1
FAISS HNSW	< 1	< 1	< 1
Swap test	993	4764	21555
Grover (hardcoded oracle)	19	20	19
Grover (quantum state prep)	3399	6916	7453
Hybrid (HNSW + swap test)	254	1134	5255

The practical consequence is that the simulator, not the algorithm, set the ceiling on how large a problem we could study, which is why the embedding dimension was capped at 256. None of these times should be read as a statement about how fast any engine would run on real quantum hardware; Section 4.6 states this constraint plainly.

4.4 IBM Hardware Sanity Check

All of the quantum results above were produced on a simulator. To confirm that the swap-test pipeline also runs on a physical quantum processor, we executed the hybrid engine once on real IBM Quantum hardware through the IBM Quantum cloud service. This run was deliberately tiny: it used two-dimensional vectors, two candidates per query, and thirty-two measurement shots, across the same twenty queries, and it consumed eighty seconds of the limited free-tier quota. We treat it as a sanity check on the execution path, not as a benchmark, and it is kept separate from the simulator results for that reason.

The run achieved an average MRR of 0.125, far below the simulator figures. This number reflects the severity of the configuration rather than any property of the method. Reducing the vectors to two dimensions destroys almost all of the information that distinguishes one image from another, two candidates leaves the task close to a coin toss, and thirty-two shots give a very noisy estimate of the swap-test overlap; real-device noise then sits on top of all three. What the run does establish is that the full hardware path works end to end: the swap-test circuit transpiles to a real device’s gate set, executes in the cloud queue, and returns measurement counts that our pipeline reads

back into a ranking. What it cannot establish is anything about accuracy or scaling, because the configuration was chosen to fit inside the free-tier quota rather than to measure retrieval quality. We therefore make no comparison between this run and the simulator results, and draw no scaling conclusion from it.

4.5 Discussion

RQ1 (Accuracy). The swap test reproduced the retrieval accuracy of classical cosine similarity once it was given enough measurement shots, and the shot budget it needed depended on how close the candidate vectors were. At dimension 256, where neighbours are well separated, it matched the classical MRR of 0.975 at every shot count we tried, including the smallest. At dimension 128 it reached classical accuracy only at the highest setting, rising to 0.883 at 2048 shots from 0.642 at 512, and at dimension 64 it remained below the classical baseline even at 2048 shots. The pattern is consistent with the swap test being a sampled estimate of vector overlap: the harder the discrimination, the more shots are required to resolve it. *Yes: the swap test reproduces classical retrieval accuracy given a sufficient shot budget, and the budget required grows as the embedding dimension, and hence the separation between neighbours, falls.*

RQ2 (Scaling). Our Grover implementation used four oracle calls per query, which is exactly the theoretical optimum $\lfloor (\pi/4)\sqrt{N'} \rfloor$ for the padded index size $N' = 32$, as set out in Section 4.2. The match was exact rather than approximate. The qualification is that we benchmarked only one dataset size, so we confirmed the formula at a single point rather than tracing the scaling curve across a range of N . *Yes: at the size we tested, the oracle-call count matched the theoretical $\lfloor (\pi/4)\sqrt{N} \rfloor$ optimum exactly, though a single dataset size cannot demonstrate the full curve.*

RQ3 (Practicality). No engine that performed its search quantumly beat the classical HNSW baseline on this dataset. HNSW reached the full brute-force accuracy of 0.829 in about five graph hops, while the standalone quantum engines were either less accurate or, in Grover’s case, dependent on a classically precomputed oracle. The one quantum-containing engine that matched the baseline, the hybrid, did so precisely because the classical index performed the search and the swap test only reranked a shortlist; its accuracy is inherited from HNSW, not produced by the quantum step. *We found no condition under which a quantum search engine outperformed the HNSW baseline on this dataset; the strongest quantum-containing engine matched it only by delegating the search itself to the classical index.*

RQ4 (Suitability). The resource accounting in Section 4.2 points to a structural

reason for the RQ3 result. Grover’s sublinear oracle count is genuine, but it measures the search after the data has been loaded into the oracle; constructing that oracle, or an equivalent quantum-accessible memory, reintroduces a cost linear in the dataset size. Vector retrieval is dominated by exactly this data-loading step, because answering a query means comparing against many stored items. Quantum advantage, by contrast, is generally argued to concentrate on compute-bound problems that do heavy processing over small inputs rather than on data-bound problems that must read a large dataset [6]. *On both our evidence and the structure of the problem, cross-modal vector search is data-bound retrieval rather than compute-bound work over small data, so it is not a natural fit for quantum advantage even setting aside the maturity of current hardware.*

4.6 Limitations

Several constraints bound what these results can claim, and we state them directly. The dataset was small, twenty images with one caption each as the query, which was deliberate: it kept every configuration fast enough to simulate and gave an unambiguous ground truth. The cost is limited statistical power and the fact that the HNSW index never had enough vectors to exercise its approximation, so its match with exact search holds for this dataset but would not necessarily hold at the scale where HNSW is normally used. The quantum engines, with the single exception of the IBM run in Section 4.4, executed on a classical simulator, so our results describe the algorithms’ behaviour and not the noise, gate errors, or latency of present-day quantum hardware. The embeddings were truncated from the model’s native 512 dimensions to at most 256 to fit the simulator’s qubit budget, which places every engine below the accuracy the full embeddings could reach, so the absolute MRR figures should be read as relative comparisons under a shared handicap rather than as the best the method can do. Grover used a classically precomputed oracle, meaning its sublinear oracle count excludes the cost of encoding the data, as discussed under RQ4. Finally, we benchmarked a single dataset size, so the Grover oracle-call scaling is confirmed at one point on its theoretical curve rather than across a range. None of these constraints changes the direction of the findings, but each marks a boundary beyond which they should not be extended.

5. SUSTAINABILITY, INCLUSIVITY, AND SOCIETAL IMPACT

This project was designed to be modest in its use of resources. The quantum engines ran on a CPU-based circuit simulator over a deliberately small dataset of twenty images, rather than on large-scale GPU training or extended quantum-hardware time, so the entire benchmark sweep completes on a single workstation; the one run on real quantum hardware consumed eighty seconds of a free-tier quota. The data raises no privacy concerns, as Flickr30k is a public research dataset of captioned images and contains no personal identifying information that we collect or process. The work is openly reproducible: the source code, the benchmark harness, and the dataset configuration are publicly available, and the harness is deterministic, so the reported results can be regenerated from scratch.

Beyond these practical points, the study makes a small contribution to how quantum computing is discussed. Public accounts often present quantum advantage as imminent and general; by measuring where quantum search helps and where it does not, and by being explicit about the data-loading cost that the headline speed-ups omit, the work offers a counterweight to that framing. In the language of the United Nations Sustainable Development Goals, the project speaks most directly to SDG 4 (quality education), as an openly available teaching artefact for quantum and classical retrieval, and to SDG 9 (industry, innovation, and infrastructure), as an honest assessment of a candidate technology’s readiness.

6. CONCLUSION

This study built a single framework in which classical, quantum, and hybrid engines answer the same cross-modal queries over the same embeddings, and used it to measure where quantum similarity search helps. The findings are consistent and, in places, sobering. Retrieval accuracy was governed by the embedding dimension rather than by whether an engine was quantum: the swap test reproduced classical accuracy once given a sufficient measurement budget (RQ1), and our Grover implementation reproduced its theoretical $\lfloor (\pi/4)\sqrt{N} \rfloor$ oracle-call count exactly at the size we tested (RQ2). No engine that performed its search quantumly outperformed the classical HNSW baseline on this dataset; the only quantum-containing engine to match it, the hybrid, did so by delegating the search to the classical index (RQ3). The structural reason is that vector retrieval is dominated by reading the dataset, a data-bound task, whereas quantum advantage concentrates on compute-bound problems over small inputs, so cross-modal search is not a natural fit for quantum methods even setting hardware maturity aside (RQ4).

This work delivers four things. The first is a reproducible benchmarking framework that runs seven engines, classical, quantum, and hybrid, behind one interface and records every result in a database. The second is an empirical verification of textbook quantum behaviour on a real retrieval task: the swap test’s convergence to the classical ranking, and Grover’s oracle-call scaling. The third is a web application that displays the engines’ rankings side by side, making the comparison inspectable rather than abstract. The fourth, and the one we consider most important, is the honest framing itself: stating plainly where quantum search does not help, and why, is a contribution in a setting where that question is often left unasked.

Several directions would extend the study. A larger dataset, drawn from the full Flickr30k set or an ImageNet subset, would exercise the HNSW approximation that twenty vectors leave untested and give the accuracy figures more statistical weight. A wider dimension sweep, and a study of variational state-preparation circuits in place of the hardcoded oracle, would probe the quantum engines closer to their intended use. Comparing against a production vector store such as pgvector’s HNSW would add a third classical baseline, and a characterisation of noise on real quantum hardware, beyond the single validation run reported here, would show how the simulated behaviour degrades on present-day devices.

REFERENCES

- [1] A. Radford, J. W. Kim, C. Hallacy *et al.*, “Learning transferable visual models from natural language supervision,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.
- [2] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [3] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [4] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 212–219.
- [5] H. Buhrman, R. Cleve, J. Watrous, and R. de Wolf, “Quantum fingerprinting,” *Physical Review Letters*, vol. 87, no. 16, p. 167902, 2001.
- [6] S. Aaronson, “Read the fine print,” *Nature Physics*, vol. 11, no. 4, pp. 291–293, 2015.
- [7] V. Giovannetti, S. Lloyd, and L. Maccone, “Quantum random access memory,” *Physical Review Letters*, vol. 100, no. 16, p. 160501, 2008.
- [8] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, “Quantum machine learning,” *Nature*, vol. 549, no. 7671, pp. 195–202, 2017.
- [9] Qiskit Contributors, “Qiskit: An open-source framework for quantum computing,” 2024, <https://www.ibm.com/quantum/qiskit>.

A. APPENDICES

Appendix A: Engine Configuration

The benchmark sweep is driven by a single configuration file, `backend/config/benchmarks.yaml`. The active configuration used for the results in Chapter 4 is reproduced below. Commenting out an entry removes it from the sweep without changing any code; the IBM hardware engine is kept commented out so that it runs only when quantum-processor time is deliberately spent.

```
classical_engines:
  - brute_force_cosine
  - faiss_flat_l2
  - faiss_hnsw_l2

quantum_engines:
  - hybrid_hnsw_swap_test
  # - hybrid_hnsw_swap_test_ibm # IBM hardware; run separately
  - qiskit_swap_test
  - qiskit_grover
  - qiskit_grover_quantum_prep

dimensions: [64, 128, 256] # CLIP output truncated to the first d
  components
shots_values: [512, 1024, 2048]
layers_values: [1]
top_k: 10

# Live /api/search runs a single configuration, chosen for
  responsiveness.
live_search:
  dimension: 128
  shots: 512
  layers: 1
  top_k: 10
```

Appendix B: Database Schema

Every benchmark run is stored as one row in the `benchmark_results` table. The schema below records the query and engine, the configuration (dimension, shots, layers), the returned and target identifiers used to compute the Mean Reciprocal Rank, and the cost measures discussed in Section 4.2. The unique constraint makes each (query, engine, dimension, shots, layers) combination idempotent, so re-running the sweep overwrites rather than duplicates.

```
CREATE TABLE benchmark_results (  
  id          BIGSERIAL PRIMARY KEY,  
  recorded_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  query_id    TEXT        NOT NULL,  
  engine_name TEXT        NOT NULL,  
  dimension   INTEGER     NOT NULL,  
  target_ids  JSONB       NOT NULL,  
  top_ids     JSONB       NOT NULL,  
  state_prep_ms DOUBLE PRECISION,  
  search_ms   DOUBLE PRECISION NOT NULL,  
  total_ms    DOUBLE PRECISION NOT NULL,  
  parameters  JSONB       NOT NULL DEFAULT '{}',  
  dataset_size INTEGER     NOT NULL DEFAULT 0,  
  circuit_depth INTEGER,  
  num_qubits  INTEGER,  
  shots       INTEGER     NOT NULL DEFAULT -1,  
  layers      INTEGER     NOT NULL DEFAULT -1,  
  oracle_calls INTEGER,  
  CONSTRAINT uq_run_key UNIQUE (query_id, engine_name, dimension, shots  
    , layers)  
);
```

Appendix C: Source Code Repository

The complete source code, the benchmark harness, the dataset configuration, and the scripts that generate the figures in this report are publicly available at <https://github.com/Mirza404/quantum-vector-search>. The repository includes the instructions needed to reproduce the dataset, run the benchmark sweep, and launch the web application described in Chapter 3.